

MID を使用した解約率予測 GLM の改良 - 基本編

Reiji Kozuma^{*} Ryoichi Asashiba[†] Yoshihiro Matsumori[‡] Hirokazu Iwasawa[§]

2026-09-25

概要

本ドキュメントは、元論文 Kozuma et al. (2026) において提案した、新たな解釈手法 MID を活用した EV 解約率前提用 GLM の改善過程における基本的な手順を、直ぐに利用可能なコード例の形で示すものである。データセットの前処理、その研究で利用したモデルの構築、およびパッケージ midr を活用した可視化の方法について述べる。

なお、使用するモデルや評価指標、解釈手法の解説については元となる論文に譲ることとし、本ドキュメントではコードを示すことによる具体的な使用方法の説明に注力する。また、EV の簡易計算や、論文のデータの再現のためのドキュメントは、別ファイルで提供する。

コードの多くは元の研究の内容を再現するためのものであり、それらの中には、初学者にとっては理解が難しいコードも存在する。そのため、初学者は、Section 2.3.2 以外の内容については単に記載されているコードを深く考えずにそのまま実行するだけでもさしあたりは問題ない。

目次

1 準備

- 1.1 パッケージの読み込み
- 1.2 データの読み込み
- 1.3 評価関数の準備

2 モデルの構築

- 2.1 Reduced Baseline GLM
- 2.2 XGBoost モデル
- 2.3 Baseline GLM / Improved GLM

3 統計学的精度評価

- 3.1 Reduced Baseline GLM
- 3.2 XGBoost モデル
- 3.3 Baseline GLM
- 3.4 Improved GLM

Acknowledgments

^{*} Daido Life Insurance Company, rktkdm@gmail.com

[†] Daido Life Insurance Company, ryoichi.asashiba@gmail.com

[‡] Sompo Japan Insurance Inc., ymatsumori1@sompo-japan.co.jp

[§] Graduate School of Accountancy, Waseda University, iwahiro@bb.mbn.or.jp

References

1 準備

1.1 パッケージの読み込み

本ドキュメントで使用するパッケージを読み込む。

```
library(midr)
library(tidyverse)
library(xgboost)
library(ROCR)
```

1.2 データの読み込み

最初に、本研究で利用したデータセット Society of Actuaries (2014) を読み込む^{*1}。

```
# install.packages("devtools")
# devtools::install_github("kasaai/insurance")
library(insurance)
df_raw <- lapse_study
```

次に、データを利用しやすいように前処理を行う。便宜上追加した ID 列を除けば、元論文で説明したとおりである。

```
# Add ID column
df_raw <- df_raw %>% mutate(ID = as.integer(rownames(df_raw)))

# Convert lapse_study_year from string "yyyy-(yyyy+1)" to integer yyyy,
# and format issue_year and duration
df_raw <- df_raw %>% mutate(
  lapse_study_year = as.integer(substr(lapse_study_year, 1, 4)),
  issue_year =
    if_else(issue_year == "Pre-1991", "-1990", issue_year) %>%
    as.ordered(),
  duration = case_when(
    duration == "6-9" ~ "06-09",
    duration == "13+" ~ "13-",
    TRUE ~ duration
  ) %>% as.ordered()
)

# Standardize post_level_premium_structure
df_raw <- df_raw %>% mutate(
  post_level_premium_structure =
    if_else(
      post_level_premium_structure == "1. Premium Jump to ART",
      "ART", "Other"
    ) %>% factor(levels = c("ART", "Other"))
)
```

^{*1} <https://github.com/kevinykuo/insurance/tree/master/data> からダウンロード可能。

```

# Format premium_jump_ratio bands
df_raw <- df_raw %>% mutate(
  premium_jump_ratio =
    case_when(
      premium_jump_ratio == "Y. Unknown" ~ "Unknown",
      premium_jump_ratio == "X. 24.01 AND UP" ~ "24.01-",
      .default = as.integer(substr(premium_jump_ratio, 4, 5)) %>%
        (function(x) sprintf("%#2d.01-%#2d.00", x, x+1))) %>%
      as.factor())

# Format categorical factor levels
levels_risk_class <-
  c("Super-Pref NS", "Pref Best NS", "Pref Resid NS", "Non-Pref NS", "Undiff NS",
    "Pref SM", "Non-Pref SM", "Undiff SM", "Agg/Unknown")
levels_premium_mode <-
  c("Annual", "Semiannual", "Quarterly", "Monthly", "Biweekly", "Unknown/Other")

df_raw <- df_raw %>% mutate(
  premium_mode = premium_mode %>% substring(4) %>%
    factor(levels = levels_premium_mode),
  risk_class = risk_class %>%
    factor(levels = levels_risk_class),
  gender = gender %>%
    factor(levels = c("M", "F"),
          labels = c("male", "female")),
  issue_age = as.ordered(issue_age),
  face_amount = face_amount %>%
    factor(levels = c("A. < 100k", "B. 100k-249k", "C. 250k-999k", "D. 1M +"),
          labels = c("-100K", "100K-250K", "250K-1M", "1M-"),
          ordered = TRUE))

# Drop unused columns and filter out zero-exposure records
df_raw <- df_raw %>% select(
  ID, lapse_study_year, policy_year,
  duration,
  issue_year, issue_age, gender, face_amount, risk_class,
  premium_mode, premium_jump_ratio, post_level_premium_structure,
  exposure_count, lapse_count,
  exposure_amount, lapse_amount
)
df_all <- df_raw %>% filter(exposure_count != 0)
df_all <- df_all %>% select(-lapse_amount) %>%
  rename(exposure = exposure_count) %>%
  rename(lapse = lapse_count)

# Calculate the observed lapse rate as the target variable
df_all <- df_all %>% mutate(lapse_rate = lapse / exposure)

prep_toint_pjratio <- function(df_raw){
  df_raw %>%

```

```
mutate(premium_jump_ratio_unknown =
  if_else(premium_jump_ratio == "Unknown", 1, 0)) %>%
mutate(premium_jump_ratio =
  fct_recode(premium_jump_ratio,
    "05" = "Unknown", "27" = "24.01-") %>%
  substr(1, 2) %>% as.integer())
}
```

後で説明するように、MID による関数分解を行うモデルは 2004-2006 保険年度で学習したもの (Validation Year は 2007) とするので、このデータを抽出して準備する。

```
year_midr <- 2006 # Validation Year - 1
df_train_2006 <- df_all %>%
  filter(lapse_study_year >= year_midr-2, lapse_study_year <= year_midr)
df_test_2006 <- df_all %>%
  filter(lapse_study_year == year_midr+1)
```

1.3 評価関数の準備

Log-loss を重み付きで計算する関数の実装を行う。本関数の引数 `mean` を `FALSE` とした場合は Log-loss の総和が、`TRUE` とした場合は加重平均値が出力される。

```
xlogy <- function(x, y){
  if_else(x == 0, 0, x * log(y))
}
calc_logloss <- function(y_pred, y_act, w = NULL, mean = T){
  losses <- - xlogy(y_act, y_pred) - xlogy(1 - y_act, 1 - y_pred)
  if(is.null(w)){
    if(mean){
      return(mean(losses))
    }else{
      return(sum(losses))
    }
  }
  if(mean){
    return(sum(w * losses) / sum(w))
  }else{
    return(sum(w * losses))
  }
}
```

続いて、ROC-AUC を重み付きで計算する関数の実装を行う。参考: [ROCR/R/prediction.R at master · ipa-tys/ROCR](#)

```
calc_auc <- function(y_pred, y_act, w = NULL, verbose = FALSE){
  predictions <- y_pred
  pred.order <- order(predictions, decreasing=TRUE)
  predictions.sorted <- predictions[pred.order]
  tp <- cumsum(y_act[pred.order] * w[pred.order])
  fp <- cumsum((1-y_act[pred.order]) * w[pred.order])
}
```

```

dups <- rev(duplicated(rev(predictions.sorted)))

tp <- c(0, tp[!dups])
fp <- c(0, fp[!dups])
cutoffs <- c(Inf, predictions.sorted[!dups])

n.pos <- sum(y_act* w)
n.neg <- sum((1-y_act) * w)
fn <- n.pos - tp
tn <- n.neg - fp
n.pos.pred <-
n.neg.pred <-

pred <- new("prediction", predictions=list(predictions),
          labels=list(c("0", "1")),
          cutoffs=list( cutoffs ),
          fp=list( fp ),
          tp=list( tp ),
          fn=list( fn ),
          tn=list( tn ),
          n.pos=list( n.pos),
          n.neg=list( n.neg),
          n.pos.pred=list(tp + fp),
          n.neg.pred=list(tn + fn))
auc <- performance(pred, "auc")
if (verbose){
  perf <- performance(pred, "tpr", "fpr")
  list(auc = auc, curve = perf)
}else{
  as.numeric(auc@y.values)
}
}

```

2 モデルの構築

2.1 Reduced Baseline GLM

本研究では複数のモデルを取り扱う。モデルの前処理もモデルによって異なることから、前処理部分をモデル固有の関数として実装することで、コードの見通しをよくする。

まず、`issue_age` に対しては、SOA のレポート P. 39 に記載された代表年齢に変換する関数を作成する。

```

prep_toint_age <- function(df_raw){
  df_raw %>%
    mutate(issue_age = case_match(issue_age,
      "0-19" ~ 9.5,
      "20-29" ~ 26.3,
      "30-39" ~ 34.7,
      "40-49" ~ 44.2,
      "50-59" ~ 53.7,
      "60-69" ~ 63.3,

```

```

    "70+" ~ 72.3
  ))
}

```

次に、SOA の作成した GLM と同様の前処理を行う関数を作成する。ただし、ここで構築するのは `face_amount` を取り除いた Reduced Baseline GLM である。

```

prepfun_soared_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
      risk_class_group = risk_class %>%
        factor(levels = levels_risk_class,
              labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
      premium_mode_group = premium_mode %>%
        factor(levels = levels_premium_mode,
              labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
                        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
      premium_jump_ratio_group = cut(
        if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
        breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%
    select(duration, post_level_premium_structure,
           issue_age, issue_age_sq, issue_age_log, risk_class_group,
           premium_mode_group, premium_jump_ratio_group, exposure, lapse)
}

```

以上の前処理を実際に適用したデータフレームを作成しておく。

```

df_train_2006_soared <- df_train_2006 %>% prepfun_soared_xy
df_test_2006_soared <- df_test_2006 %>% prepfun_soared_xy

```

最後に、学習データに対して `glm()` 関数を適用し、モデルを構築する。`duration : post_level_premium_structure` の記述は、これらについて層化したモデルとしていることを表す。

`duration : post_level_premium_structure` の指定は計画行列が巨大になることから、層ごとに別々の GLM を構築するよりも計算時間がかかる。この観点で元論文で掲載した精度評価には、層ごとに別々の GLM を構築したものを使用している。両者は本質的に同等なモデルであるが、1 件も解約が発生しないケースの予測においてわずかに出力される確率値が異なっている。従って、本ドキュメントによる実装は元論文の結果を厳密には再現しない。

```

model_soared_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  ↪ post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_soared)

```

学習データで現れなかったグループがテストデータには現れることがあるという点に注意が必要である。そのようなデータに対しても予測できるよう、テストデータにも前処理が必要となる。今回使用する年度のデータに対しては、次のような前処理が必要となるので、それを関数として作成する。

```

prepfun_soared_xy_test_2006 <- function(df_tmp){
  df_tmp %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "10" &
        premium_jump_ratio_group == "[20,99)", "[16,20)",
        premium_jump_ratio_group)
  ) %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "12" &
        premium_jump_ratio_group == "[16,20)", "[12,16)",
        premium_jump_ratio_group)
  ) %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "13-" &
        premium_jump_ratio_group == "[12,16)", "[10,12)",
        premium_jump_ratio_group)
  ) %>% mutate(
    risk_class_group = if_else(
      post_level_premium_structure == "Other" & duration == "11" &
        risk_class_group == "Super-Pref NS", "NS",
        risk_class_group)
  )
}

```

2.2 XGBoost モデル

次に、GLM の改良に利用するブラックボックスモデルとして XGBoost による予測モデルを構築する。

XGBoost のモデルにデータを投入する際は、事前に数値型の計画行列に変換したうえで、`xgb.DMatrix()` 関数を用いてデータを変換する必要がある。またそのさい、すべての行で同じ数値となっている列が存在するとエラーになってしまうため、それを取り除いておく。

```

get_mm <- function(df){
  df <- df %>%
    select(setdiff(names(df),c(
      "ID", "lapse_study_year", "policy_year", "issue_year",
      "exposure_amount", "lapse", "exposure", "lapse_rate")))) %>%
    prep_toint_pjratio
  model.matrix(~ ., df)
}
df_train_2006_mm <- df_train_2006 %>% get_mm
constant_cols_2006 <- which(apply(df_train_2006_mm, 2, function(col) all(col == col[1])))
dm_train_2006 <- xgb.DMatrix(
  data = df_train_2006_mm[, -constant_cols_2006, drop = FALSE],
  label = df_train_2006 %>% pull(lapse_rate),
  weight = df_train_2006 %>% pull(exposure))

```

`xgb.cv()` 関数などのパラメータ `params` に与えるパラメータについては次のとおりである。パラメータ `objective` に `"reg:logistic"` を指定し、ロジスティック損失関数を `objective` として確率を出力させる。また、過学習防止のために木の複雑さを設定する必要がある。その交差検証での判定基準として、パラメータ

eval_metric を "logloss" に指定する。加えて、ハイパーパラメータを設定する。本研究では、tune パッケージを利用して事前にチューニングした^{*2}。

```
params_xgb <- list(  
  objective = "reg:logistic",  
  eval_metric = "logloss", #Used in CV  
  # Hyper parameters. Determined by Bayesian optimization using the "tune" package.  
  gamma = 2.966439e-08, #loss_reduction  
  eta = 0.02, #learn_rate  
  max_depth = 10, #tree_depth  
  min_child_weight = 10, #min_n  
  subsample = 0.9976594, #sample_size  
  colsample_bynode = 6/31 #mtry / count of predictors  
)
```

以上の準備のもと、xgb.cv() 関数により交差検証で木の複雑さ nrounds を決定し、xgb.train() 関数で予測モデルの構築を行う。

```
set.seed(2025)  
# Determine the optimal number of boosting rounds by CV  
cv_xgboost_2006 <- xgb.cv(data = dm_train_2006, nrounds = 2000, nfold = 4,  
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)  
nrounds <- cv_xgboost_2006$early_stop$best_iteration  
  
# Fit model  
set.seed(2025)  
model_xgboost_2006 <- xgb.train(data = dm_train_2006, nrounds = nrounds,  
  params = params_xgb)
```

この XGBoost が GLM よりも精度の高いモデルであることは別途検証が必要であるが、ここでは記載を省略する。

2.3 Baseline GLM / Improved GLM

ここまでで構築した XGBoost の予測結果を MID を用いて観察し、観察結果を参考にして GLM を改良する。以下では、Section 2.1 で構築した、意図的に face_amount を除外したモデル（Reduced Baseline GLM）を改良の出発点とする。

つづく節では、改良の手順の一部をコード例を示しながら説明する。

2.3.1 関数分解と特徴量重要度

関数分解を行うモデルの対象となるデータを用意する。ここで、本研究では検証年度に応じて複数のモデルを構築しているが、MID を観察するモデルは 2004-2006 保険年度で学習したモデルとし、関数分解に用いるインスタンスも、その学習データとする。これは、SOA の GLM と XGBoost の間の予測の差が一定程度大きく、かつ解約の予測が外れやすい時期にはまだ差し掛かっていないことによる^{*3}。

^{*2} learning rate は 0.02 に固定した。その他のハイパーパラメータは、2005 年から 2007 年の学習データを用いた 4 分割交差検証によって一度だけ調整され、ローリングウィンドウ全体を通じて固定した。一方、ブースティングの反復回数は、各ウィンドウについて、そのウィンドウに対応する学習データ内での 4 分割交差検証により個別に決定した。

^{*3} 元論文に記載する EV 変動要因分析による前提実績差異を確認すると、2008 保険年度以降は XGBoost でも差異が大きくなっており、将来に向かっての解約の予測が困難であることを示唆している。

重みを指定するためのデータや、予測に使用する XGBoost モデル固有の関数 (`pred.fun`) を用意する必要がある^{*4}ので、それをここで定義する。

```
df_train_2006_xw <- df_train_2006 %>%
  select(-ID, -lapse_study_year, -policy_year, -issue_year, -exposure_amount, -lapse,
    ↪ -lapse_rate)
df_train_2006_w <- df_train_2006_xw %>% pull(exposure)
predfun_xgb <- function(model, newdata){
  df_mm <- newdata %>% get_mm
  df_mm <- df_mm[, -constant_cols_2006, drop = FALSE]
  predict(model, df_mm)
}
```

まず最初のステップとして、XGBoost の予測値を関数分解するため、`midr::interpret()` 関数を実行する^{*4}。MID of order 2 までの分解が実行可能だが、今回のケースでは分解のときに、すべての交互作用項を用意すると singular fit の回避が困難であるため、一部の交互作用項を省略した形の分解を行う。

```
mid_xgb <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+
        face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_xgboost_2006, pred.fun = predfun_xgb, link = "logit",
  # Penalty factor for pseudo-smoothing; helps avoid singular fits.
  lambda = 0.01,
  verbosity = 0 # Stop when a singular fit is encountered.
)
```

可視化のステップに入る前に、見た目を整えるために、幾つかの準備を行う。これは分析上必須ではない。

```
# Display border lines.
theme_bw16 <- theme_bw(base_size = 16)
# Display the text on the X-axis vertically.
theme_axisvert <- theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
theme_set(theme_bw16 + theme_axisvert)
# Set default color theme (ggmid's argument "theme").
options(midr.sequential = "Oslo")
# Modify appearance of scatter plots.
update_geom_defaults("point",list(alpha = 0.5, size = 2.0, shape = 16))
# Utility function to add labels
add_label <- function(data){
  geom_label(
    data = data, aes(x = x, y = y, label = label),
    size = 5, fill = "white", colour = "black",
    label.size = 0.1, label.padding = unit(0.1, "lines"), fontface = "bold"
  )
}
```

^{*4} midr パッケージの使用方法的詳細は <https://cran.r-project.org/web/packages/midr/readme/README.html> を参照。

次に特徴量とそれらの間の交互作用の重要度を可視化する。結果は Figure 1 のとおり。

```
mid_xgb %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap")
```

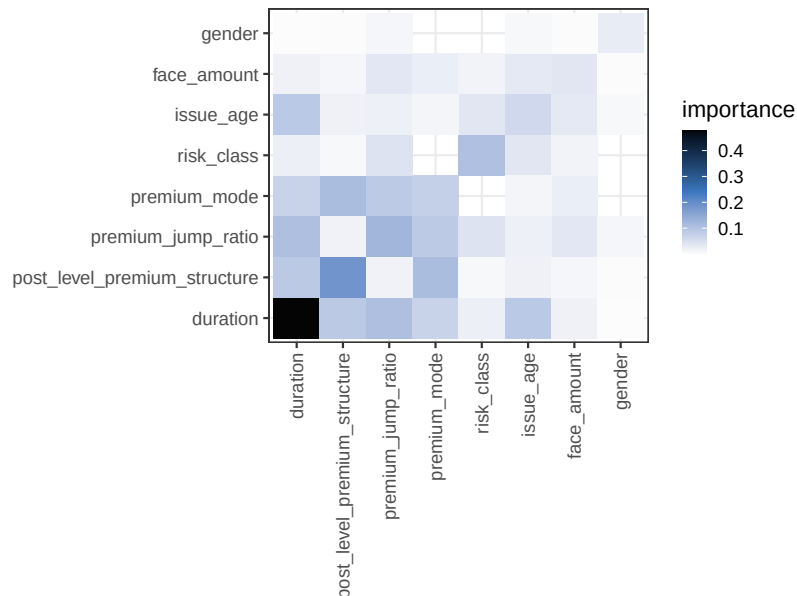


Figure1: Feature Effect Importance

関数分解を行う対象を Reduced Baseline GLM と XGBoost の予測値の差に変更することで、両者の予測の差異がどの説明変数に関係して現れるのかを可視化することもできる。結果は Figure 2 のとおり。

いずれも、分解の対象となる予測値が確率の値そのものではなく線形予測子のもの (logit 変換を介した後の数値) であることに注意する。

```
fn_logit <- function(p){log(p/(1-p))} # link function
fn_expit <- function(x){1/(1+exp(-x))} # inverse of the link function

# pred.fun for the SOA GLM model
predfun_soared_2006 <- function(model, newdata){
  predict(model,
    newdata %>% mutate(lapse = 0) %>% #prevent error in prepfun_soared_xy
    prepfun_soared_xy %>% prepfun_soared_xy_test_2006,
    type = c("response"))
}

# pred.fun for "difference" models
predfun_diff <- function(model, newdata){
  p_target <- model$predfun_glm(model$model_glm, newdata)
  p_base <- model$predfun_xgb(model$model_xgb, newdata)
  fn_expit(fn_logit(p_target) - fn_logit(p_base)) #apply the link function
}
```

```

# Create the "difference" model
model_diff <- list()
model_diff$model_xgb <- model_xgboost_2006
model_diff$model_glm <- model_soared_2006
model_diff$predfun_xgb <- predfun_xgb
model_diff$predfun_glm <- predfun_soared_2006

# Execute midr::interpret for the "difference" model
mid_diff <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_diff, pred.fun = predfun_diff,
  link = "logit", lambda = 0.01,
  verbosity = 0
)

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("post_level_premium_structure",
        "risk_class", "premium_mode", "premium_mode", "premium_mode",
        "issue_age", "risk_class", "premium_jump_ratio", "issue_age",
        ↵ "premium_jump_ratio", "gender"),
  y = c("post_level_premium_structure",
        "risk_class", "premium_mode", "premium_jump_ratio",
        ↵ "post_level_premium_structure",
        "issue_age", "issue_age", "risk_class", "face_amount", "face_amount", "gender"),
  label = c("X", "A-2", "A-3", "A-3", "A-4", "A-5", "A-6", "A-7", "A-8", "A-9", "A-10")
))

```

```

#display feature effect imporatanace plot
mid_diff %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap") + add_label_tmp

```

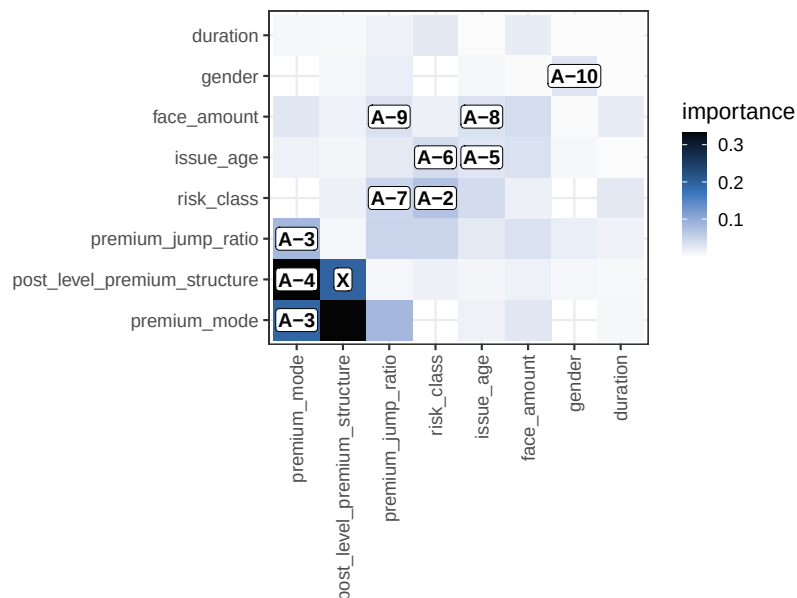


Figure2: Feature Effect Importance of the difference model

今回の GLM は層化されているため、ある層に特化した分析を行うことも考えられる。たとえば、レポートで GLM の構築対象とした `duration == 10`, `post_level_premium_structure == "ART"` にデータを限定して関数分解を行うと Figure 3 のとおり。

```
# Create filtered data.
df_train_2006_xw_d10ART <- df_train_2006_xw %>%
  filter(duration == 10, post_level_premium_structure == "ART")
df_train_2006_w_d10ART <- df_train_2006_xw_d10ART %>% pull(exposure)

# midr::interpret.
mid_xgb_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
  model = model_xgboost_2006, pred.fun = predfun_xgb,
  link = "logit", lambda = 0.01, verbosity = 0
)
mid_diff_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
  model = model_diff, pred.fun = predfun_diff,
  link = "logit", lambda = 0.01, verbosity = 0
)

# Display feature effect importance plot.
```

```

mid_xgb_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("XGBoost")

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("premium_jump_ratio", "face_amount", "risk_class", "premium_mode", "risk_class"),
  y = c("premium_jump_ratio", "face_amount", "risk_class", "premium_jump_ratio",
        "premium_jump_ratio"),
  label = c("Y", "B-1", "B-2", "B-3", "B-7")
))

# Display feature effect importance plot.
mid_diff_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("GLM - XGBoost") + add_label_tmp

```

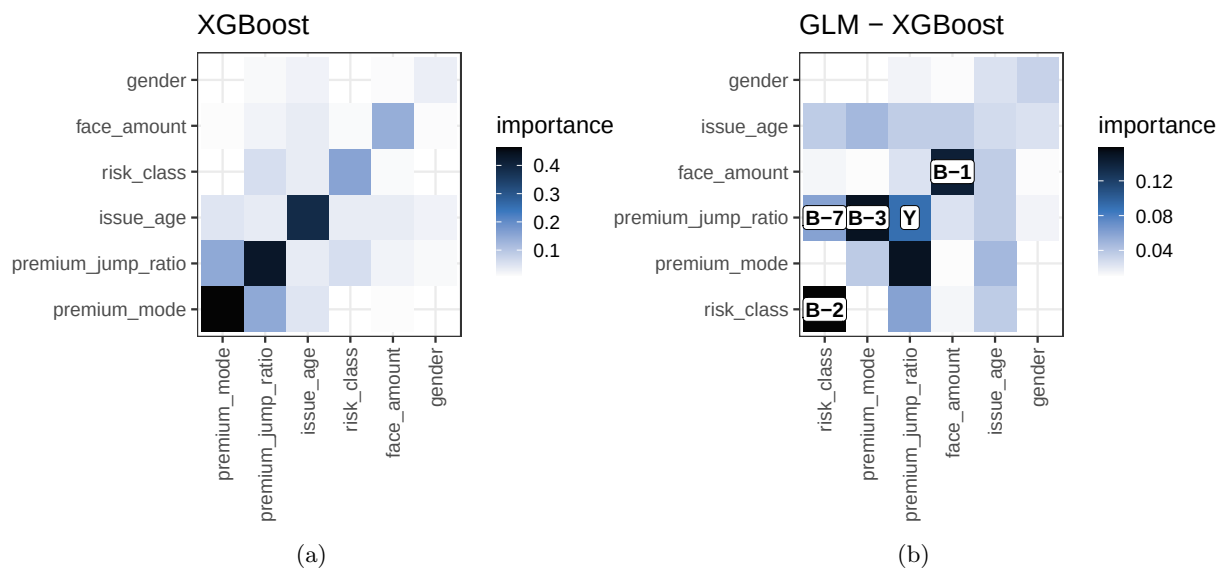


Figure3: Feature Effect Importance in Duration = 10, ART

2.3.2 モデル改良

GLM と XGBoost の予測結果の差を関数分解して得られる特徴量重要度の図, すなわち Figure 2 と Figure 3b を確認し, 差異が大きいと考えられる説明変数に着目して分析を行う.

なお, 差異が大きい説明変数全てで, GLM のモデルの改良点が見つかるとは限らない. たとえば, Figure 2 の X によると, post_level_premium_structure の主効果で差異が大きいという表示があるが, GLM はこの説明変数で層化されており, この主効果の差をモデルに明示的に反映することは出来ない. またたとえば, Figure 3b の Y によると, premium_jump_ratio の主効果で差異が大きいという表示がある. Reduced Baseline GLM ではこの説明変数についてベニングの上でダミー変数化している. そのベニング方法の調節が

考えられる程度で、関数分解の形からモデル改善に有用な直接的な示唆を得るのは難しいと考えられる。

以上を除き、おおむね差異が大きいように思われる点から順番に吟味していく。

本ドキュメントでは、冒頭の3項目について改良手順を述べる。他の詳細 (I-4 以降) は割愛する。

I-1: face_amount の主効果

Figure 3b の B-1 によると, face_amount の主効果で差異が大きいという表示がある。この説明変数は Reduced Baseline GLM では採用されていないが, この図からは解約率との何らかの関係性が示唆される。そこで, XGBoost の予測値の関数分解で得られた関数を確認する。

関数分解で得られた関数をプロットするには, `midr::ggmid()` 関数を使用する。

`duration == 10`, ART における, face_amount の主効果の関数項をプロットすると Figure 4 のとおりで, 高額契約ほど解約率が高いことが示唆される。

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw, term = "face_amount")
```

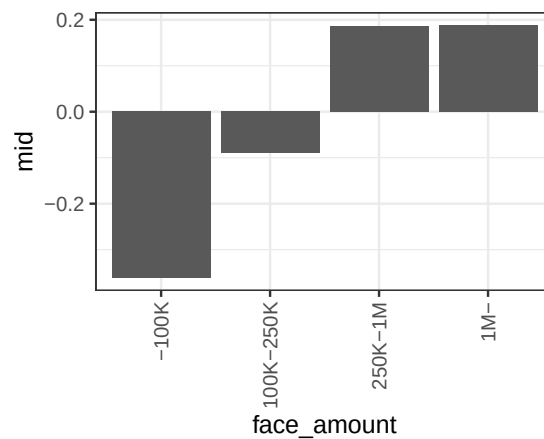


Figure4: Main Effect Plot of face_amount

Figure 3b および Figure 4 は `duration == 10`, ART に限った分析を示している。そこで, 他の duration での様子も同時に可視化するため, face_amount と duration の2変数を組み合わせた効果を可視化すると Figure 5 のようになる。

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),  
  term = "face_amount:duration", main.effects = TRUE,  
  type = "compound") #add sample points to the plot
```

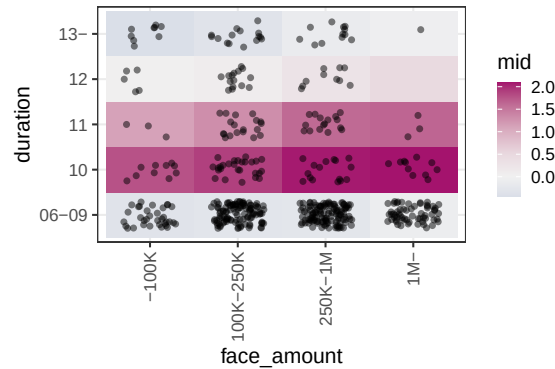


Figure5: Main Effect + Interaction Plot of duration and face_amount

Figure 5 により, $\text{duration} \leq 9$ では高額契約ほど解約率が低く, $\text{duration} \geq 10$ ではその逆であることがわかる. GLM に説明変数として `face_amount` を追加することで, 精度改善に貢献するものと考えられる.

I-2: risk_class の主効果

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
  term = "risk_class:duration", main.effects = TRUE,
  type = "compound")
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
  term = "premium_mode:duration", main.effects = TRUE,
  type = "compound")
```

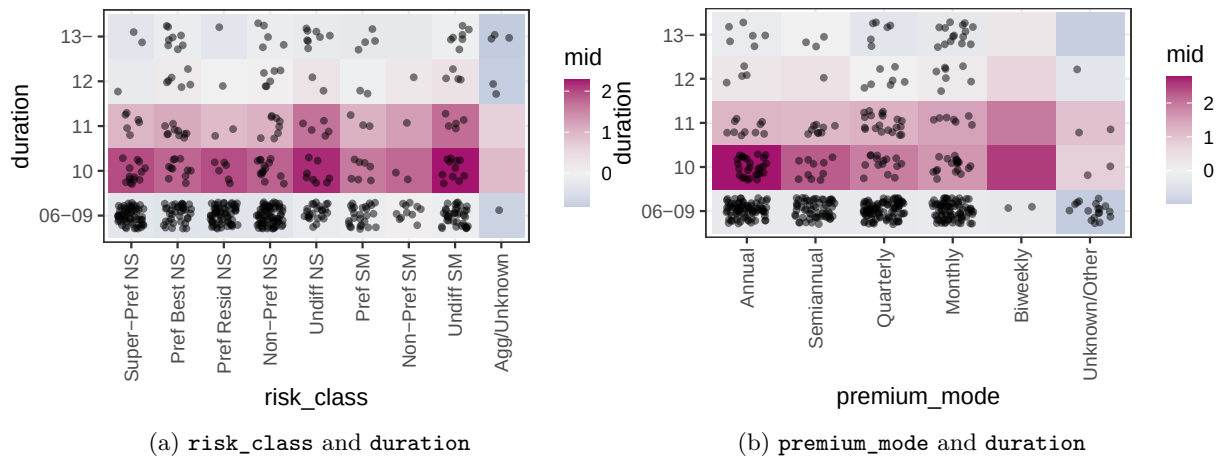


Figure6: Main Effects + Interaction Plots

Figure 2 の A-2 および Figure 3b の B-2 によると, `risk_class` の主効果で差異が大きいという表示がある. そこで I-1 と同じように `risk_class` と `duration` の 2 変数を組み合わせた効果を可視化すると Figure 6a のとおり. GLM は "Super-Pref NS", それ以外の "NS", その他の 3 区分でのみ区分けしていたが, "Undiff NS", "Undiff SM" はそれ以外の "NS", "SM" とは異なる傾向を示している. さらに,

"Agg/Unknown" はどれも異なる傾向を示している。加入時の健康状態という意味では、"Undiff" が付されているものはそうでないものは一定程度類似しているはずであるが、実際は解約率に影響が現れる形で契約者の性質が異なる（例えば保険会社、地域、チャネル等の特性の差異がある）と考えられる。"Agg/Unknown" も同様に、ある特殊な契約群団を特定できる要素となっている可能性がある。健康状態そのものが解約動向にどのような影響を及ぼすかは判断の難しいところではあるが、この図を観察する限りでは解約率には差があるため、これを区分することは解約率予測精度向上に資するものと考えられる。そこで、GLM でも "Undiff NS", "Undiff SM", "Agg/Unknown" は独立した level として設定する変更を採用する^{*5}。

I-3: premium_mode と premium_jump_ratio の交互作用

Figure 2 の A-3 および Figure 3b の B-3 によると、premium_mode と premium_jump_ratio の交互作用で差異が大きいという表示がある。この交互作用の確認の前に、premium_mode の主効果について検討するため、risk_class と duration の 2 変数を組み合わせた効果を可視化すると Figure 6b のとおり。GLM は "Semiannual", "Quarterly" を統合してひとつの level としていたが、実際は解約率に差が認められるため、これらを分離する変更を採用する。一方、"Biweekly" は "Monthly" と同じ区分となっているが、群団として非常に小さいため区分の分離は行わない。

次に、premium_mode と premium_jump_ratio の交互作用の確認のため、duration == 10, ART にてこの 2 変数の効果をプロットすると Figure 7 のとおり。Figure 7a は主効果の関数値を加算している一方、Figure 7b は交互作用の関数値のみをプロットしている。

グラフに主効果を加算するかどうかは、引数 main.effects によって制御できる。

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode", main.effects = TRUE,
                               type = "compound") +
  ggtitle("Duration = 10, ART, Main Effects + Interaction")
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode",
                               type = "compound") +
  ggtitle("Duration = 10, ART, Interaction Only")
```

Figure 7b によると、"Quarterly" の区分では、premium_jump_ratio が高くなるほど解約率が高くなる、といった関係性が弱くなることが示唆されている。そこで、duration == 10 かつ "Quarterly" の場合にのみ、premium_mode と premium_jump_ratio の交互作用項を追加する変更を採用する^{*6}。

2.3.3 モデル構築

2.3.3.1 Baseline GLM の構築

修正 I-1 により face_amount を復元した「Baseline GLM」、つまり SOA が構築したものと同等のモデルを次のとおり構築する。前処理のための関数をごく一部修正するだけでよい。

^{*5} 実際は、層化によって各層の学習データが小さくなっており、対象の level に属するデータが非常に少ないと過学習に陥る。予測精度を向上させるため、一定程度の規模がある場合のみ区分することとした。区分の判断基準は次のとおりで、閾値は予測精度が向上するかによって決定した。(i) If "Super-Pref NS" is smaller than 50, then merge "Super-Pref NS" and "NS". (ii) If "Undiff NS" or NS is smaller than 700, then merge "Undiff NS" and "NS". (iii) If "Agg/Unknown" is smaller than 100, then merge "Undiff SM" and "Agg/Unknown". (iv) If "SM" is smaller than 700, then merge "Undiff SM" and "SM".

^{*6} 関数項が過度に複雑にならないよう、この交互作用項では premium_jump_ratio のビンングを breaks = c(0,1,2,5,99) とする。また、学習データに実在データが無いビンでの予測が必要な場合、学習データがある最も近い premium_jump_ratio のビンに振り分ける。そのさい、より高いビンがあればそちらを、そうでなければより低いビンを採用する。

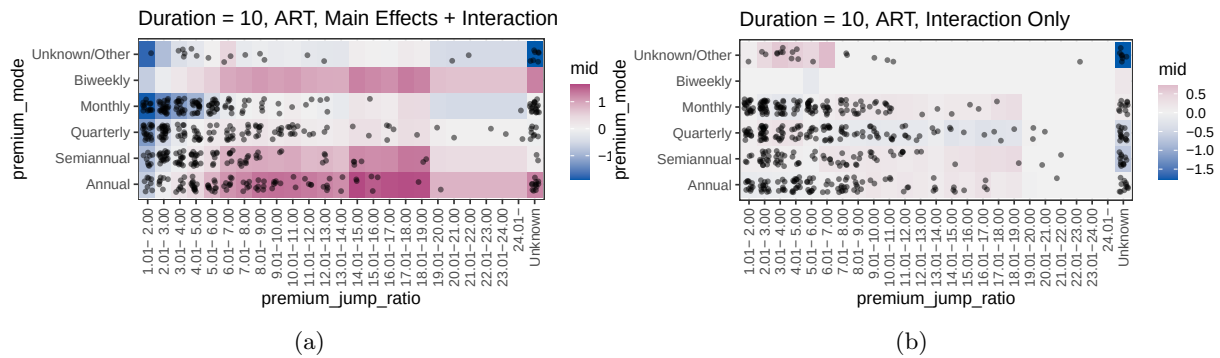


Figure7: Effects of premium_jump_ratio and premium_mode

```
prepfun_soa_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
      risk_class_group = risk_class %>%
        factor(levels = levels_risk_class,
              labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
      premium_mode_group = premium_mode %>%
        factor(levels = levels_premium_mode,
              labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
                        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
      premium_jump_ratio_group = cut(
        if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
        breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%
    select(duration, post_level_premium_structure,
           issue_age, issue_age_sq, issue_age_log, risk_class_group,
           premium_mode_group, premium_jump_ratio_group,
           face_amount, #I-1
           exposure, lapse)
}
```

```
df_train_2006_soa <- df_train_2006 %>% prepfun_soa_xy
df_test_2006_soa <- df_test_2006 %>% prepfun_soa_xy

model_soa_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  ⇨ post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_soa)
```

2.3.3.2 Improved GLM の構築

元論文における 10 個の改良を全て施したモデル「Improved GLM」の構築方法は次のとおり。改良の大部分は、前処理のための関数の中に実装される。

```

prepfun_mod_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      premium_mode_oo = #I-4
      if_else(
        duration >= "10" & post_level_premium_structure == "Other" &
        premium_mode == "Unknown/Other", 1, 0),
      post_level_premium_structure = if_else( #I-4
        premium_mode_oo == 1, "ART", post_level_premium_structure),
      premium_mode = if_else( #I-4
        premium_mode_oo == 1, "Monthly", premium_mode),
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
      issue_age_70 = if_else(issue_age >= 70 & duration == "06-09", 1, 0), #I-5
      issue_age_d10ART = if_else( #I-8
        (duration == "06-09" | duration == "10") & post_level_premium_structure == "ART",
        issue_age, 0
      ),
      # risk_class_group = risk_class %>% #cf) non #I-2
      # factor(levels = levels_risk_class,
      #       labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
      risk_class_group = risk_class %>% #I-2
      factor(levels = levels_risk_class,
            labels = c("Super-Pref NS", "NS", "NS", "NS", "Undiff NS",
                      "SM", "SM", "Undiff SM", "Agg/Unknown")),
      risk_class_group = if_else( #I-2 (adjusted to 2006-train)
        risk_class_group == "Undiff NS" &
        (duration != "06-09" | post_level_premium_structure != "ART"),
        "NS",
        if_else(
          risk_class_group == "Undiff SM" &
          (duration >= "11" & post_level_premium_structure == "ART" |
           duration <= "12" & post_level_premium_structure == "Other") |
          risk_class_group == "Agg/Unknown" &
          (duration <= "11" & post_level_premium_structure == "ART"),
          "SM",
          risk_class_group
        )
      ),
      risk_class_int = factor( #I-6 (adjusted to 2006-train)
        if_else(
          (risk_class == "Undiff SM" | risk_class == "Non-Pref NS") & duration == "06-09" |
          (risk_class == "Non-Pref SM" | risk_class == "Undiff NS" ) &
          duration == "06-09" & post_level_premium_structure == "ART" |
          risk_class == "Non-Pref NS" &
          duration == "10" & post_level_premium_structure == "ART",
          risk_class, "Super-Pref NS"),
          levels = levels_risk_class),
      premium_mode_group = premium_mode %>% #cf) non #I-3
      factor(levels = levels_premium_mode,
            labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",

```

```

        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group = premium_mode %>% #I-3
  factor(levels = levels_premium_mode,
        labels = c("Annual", "Semiannual", "Quarterly",
                    "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group_q = if_else( #I-3
  duration == "10" & premium_mode == "Quarterly", 1, 0),
premium_jump_ratio_group = cut(
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE),
premium_jump_ratio_group2 = cut( #I-3
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 5, 99), right = FALSE),
premium_jump_ratio_rc_NPNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration >= "11" & post_level_premium_structure == "Other") &
    risk_class == "Non-Pref NS", premium_jump_ratio, 0),
premium_jump_ratio_rc_PBNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration == "12" & post_level_premium_structure == "Other") &
    risk_class == "Pref Best NS" & (duration >= "10"), premium_jump_ratio, 0),
face_amount_d09PrUnHi = if_else( #I-9 (adjusted to 2006-train)
  duration == "06-09" & post_level_premium_structure == "ART" &
    premium_jump_ratio_unknown == 1 & face_amount == "1M-",
  1, 0)
) %>%
select(duration, post_level_premium_structure,
       issue_age, issue_age_sq, issue_age_log, risk_class_group,
       issue_age_70, #I-5
       issue_age_d10ART, #I-8
       risk_class_int, #I-6
       premium_mode_group, premium_jump_ratio_group,
       premium_mode_oo, #I-4
       premium_jump_ratio_group2, premium_mode_group_q, #I-3
       premium_jump_ratio_rc_NPNS, premium_jump_ratio_rc_PBNS, #I-7
       face_amount, #I-1
       face_amount_d09PrUnHi, #I-9
       gender, #I-10
       exposure, lapse)
}

```

モデル構築のステップでは、いくつかの交互作用項を明示的に追加するのみである。前処理で追加された説明変数は formula の . 指定で自動的にモデルに追加される。

```

df_train_2006_mod <- df_train_2006 %>% prepfun_mod_xy
df_test_2006_mod <- df_test_2006 %>% prepfun_mod_xy

model_mod_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age
     -premium_jump_ratio_group2 -premium_mode_group_q #I-3
     + premium_jump_ratio_group2 : premium_mode_group_q #I-3

```

```

+ risk_class_int * issue_age #I-6
+ issue_age_d10ART : face_amount #I-8
- 1) : duration : post_level_premium_structure,
family = quasibinomial(),
data = df_train_2006_mod)

```

3 統計学的精度評価

3.1 Reduced Baseline GLM

Reduced Baseline GLM の評価指標は次のとおり。関数 `calc_logloss()` の引数 `mean` を `FALSE` とした場合は Log-loss の総和が、`TRUE` とした場合は加重平均値が出力される。

```

pred_soared <- model_soared_2006 %>%
  predict(df_test_2006_soared %>% prepfun_soared_xy_test_2006, type = c("response"))

calc_logloss(
  pred_soared,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2988156

```

[1] 0.2988156

```

calc_logloss(
  pred_soared,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#238153.1

```

[1] 238153.1

```

calc_auc(
  pred_soared,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.8065347

```

[1] 0.8065347

3.2 XGBoost モデル

XGBoost の評価指標は次のとおり。元論文の Statistical Predictive Performance の節で示した数値 (Validation Year は 2007) が再現される。

```

df_test_2006_mm <- df_test_2006 %>% get_mm
df_test_2006_mm <- df_test_2006_mm[, -constant_cols_2006, drop = FALSE]
pred_xgb <- model_xgboost_2006 %>% predict(df_test_2006_mm)

```

```
calc_logloss(
  pred_xgb,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2935507
```

[1] 0.2935507

```
calc_logloss(
  pred_xgb,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#233957.1
```

[1] 233957.1

```
calc_auc(
  pred_xgb,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.820604
```

[1] 0.820604

3.3 Baseline GLM

Baseline GLM の評価指標は次のとおり。Section 2.1 で注意したように、元論文の Statistical Predictive Performance の節で示した数値からは、Log-loss で僅かにずれが生じる。

```
pred_soa <- model_soa_2006 %>%
  predict(df_test_2006_soa %>% prepfun_soared_xy_test_2006, type = c("response"))

calc_logloss(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2984396
```

[1] 0.2984396

```
calc_logloss(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#237853.5
```

```
[1] 237853.5
```

```
calc_auc(  
  pred_soa,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.8073218
```

```
[1] 0.8073218
```

3.4 Improved GLM

Improved GLM の評価指標は次のとおり。Section 2.1 で注意したように、元論文の Statistical Predictive Performance の節で示した数値からは、Log-loss で僅かにずれが生じる。なお、Baseline GLM と比べて Log-loss ではわずかに悪化している点は元論文と同様である。

```
pred_mod <- model_mod_2006 %>%  
  predict(df_test_2006_mod %>% prepfun_soared_xy_test_2006, type = c("response"))  
  
calc_logloss(  
  pred_mod,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = TRUE  
)#0.2984574
```

```
[1] 0.2984574
```

```
calc_logloss(  
  pred_mod,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = FALSE  
)#237867.7
```

```
[1] 237867.7
```

```
calc_auc(  
  pred_mod,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.8090849
```

```
[1] 0.8090849
```

Acknowledgments

本研究の実施にあたり、公表データ (SOA 10-Year Level Premium Term Lapse Study) を提供いただいた Society of Actuaries (SOA) に深謝いたします。なお、本論文の作成プロセスにおける英文表現の推敲および

コード構築の補助にあたっては、適切な人間による確認と責任のもとで生成 AI ツール（Gemini / ChatGPT 等）を活用いたしました。

References

- Kozuma, R., Asashiba, R., Matsumori, Y., & Iwasawa, H. (2026). *Enhancing GLM-based lapse rate models for EV calculation using interpretable machine learning*.
- Society of Actuaries. (2014). *2014 post level term lapse & mortality report*. <https://www.soa.org/resources/experience-studies/2014/research-2014-post-level-shock/>